Access Process

OS directs disk through three-stage access process —

- Stage 1: Seek Time
  - Positioning the head over the proper track is called a seek
  - Average seek time is sum of all possible seeks divided by count
  - Advertised times range from 3 ms to 13 ms
  - Actual average often lower due to locality of references
- Stage 2: Rotational Latency
  - Wait time for desired sector to rotate under head
  - Termed rotational latency or rotational delay
  - Average latency is time for half a rotation (Approximately 5.6 ms at 5400 RPM)
- Stage 3: Transfer Time
  - Time to transfer a block of bits (150 to 250 MB/sec in 2020)
  - Function of sector size, rotation speed, and recording density
  - Built-in controller cache allows higher transfer rates

Disk Access Example

- Calculate the average read time for a disk access given the following performance parameters:
  - Data size: 512 bytes
  - Rotation speed: 15,000 RPM
  - Transfer rate: 100 MB/s
  - Average seek time: 4 ms
  - Controller overhead: 0.2 ms
- Solution:
  - Rotational latency, $T_{rotation} = 0.5 \times \frac{1}{15{,}000/60} ms = 0.5 \times \frac{1}{250} ms = 2ms$
  - Transfer time, $T_{transfer} = \frac{512}{100 \times 10^6} s = 5.12 \times 10^{-6} s = 0.005ms$
  - Total average read time,
    $T_{access} = T_{seek} + T_{rotation} + T_{transfer} + T_{controller} = (4 + 2 + 0.005 + 0.2)ms = 6.205ms$
- If actual average seek time is $1ms$, average read time would be $3.205ms$

Total Bits in a Cache

- Total cache size depends on the size of the data and the address
- Cache must store data, tags, and valid bits
- For a direct-mapped cache with 32-bit addresses:
  - Cache size is 2^n blocks (n bits for the index)
  - Block size is 2^m words (m bits for word offset, 2 for byte part)
- Tag field size is $32 - (n + m + 2)$ bits
- Total bits equal

  $$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$
- Naming conventions specify cache size based only on the data portion

```
4 words per block
2^2 words ->
00 -> word1
01 -> word2
10 -> word3
11 -> word4

00 -> byte1
01 -> byte2
10 -> byte3
11 -> byte4
```

Example: Bits in a Cache

- Task: Find total bits for a 16 KiB direct-mapped cache with 64-bit addresses
- 16 KiB of data is equivalent to 4096 (2^{12}) words
- Four-word blocks (2^2) mean there are 1024 (2^{10}) blocks in the cache
- Each block has $4 \times 32 = 128$ bits of data
- Tag size is $64 - 10$ (index) $- 2$ (word) $- 2$ (byte) $= 50$ bits
- Complete cache size is $2^{10} \times (128 + 50 + 1) = 179$ Kibibits
- Total physical size is roughly 1.4 times the required data storage

```
1 kibibit = 1024
bit (2^10 bits)
```

```
cache line-
1 + 50 + 128
179 / 128 = 1.398
```

Example: Mapping an Address

- Task: Map byte address 1200 in a 64-block cache with 16-byte blocks
- Block address equals $\lfloor \text{Byte address}/\text{Bytes per block} \rfloor$
- Block address for 1200 is $\lfloor 1200/16 \rfloor = 75$
- Cache block number equals the block address modulo total cache blocks
- Cache block maps to 75 **mod** 64 = 11
- This specific block holds all addresses between 1200 and 1215

word1 1200 - 1203
word2 1204 - 1207
word3 1208 - 1211
word4 1212 - 1215

Measuring Cache Performance

- CPU time can be divided into two components —
  - CPU execution clock cycles: Normally includes cache hit time
  - Memory-stall clock cycles: Mainly from cache misses

$$\boxed{\textit{CPU time} = (\textit{CPU execution cycles} + \textit{Memory-stall cycles}) \times \textit{Clock cycle time}}$$

- With simplified assumptions, we can define memory-stall clock cycles as —

$$\boxed{\textit{Memory-stall cycles} = (\textit{Read-stall cycles} + \textit{Write-stall cycles})}$$

Where,

- $\textit{Read-stall cycles} = \frac{\textit{Reads}}{\textit{Program}} \times \textit{Read miss rate} \times \textit{Read miss penalty}$
- $\textit{Write-stall cycles} = \left(\frac{\textit{Writes}}{\textit{Program}} \times \textit{Write miss rate} \times \textit{Write miss penalty}\right) + \textit{Write buffer stalls}$

Memory-stall Clock Cycles

- Considering write buffer stalls are negligible, in most write-through organizations, the read and write miss penalties are same.

$$\begin{aligned}\therefore \textit{Memory-stall cycles} &= \frac{\textit{Memory access}}{\textit{Program}} \times \textit{Miss rate} \times \textit{Miss penalty} \\ &= \frac{\textit{Instructions}}{\textit{Program}} \times \frac{\textit{Misses}}{\textit{Instruction}} \times \textit{Miss penalty}\end{aligned}$$

Cache Performance Example

- Given,
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty is = 100 cycles
  - Base CPI without stalls (ideal cache) = 2
  - Load and stores are 36% of instructions
- Solution:
  - Memory miss cycles count in terms of I instructions:
    - I-cache: $I \times 2\% \times 100 = 2.00I$
    - D-cache: $I \times 36\% \times 4\% \times 100 = 1.44I$
  - Total stalled cycle count $= 2.00I + 1.44I = 3.44I$
  - Total CPI including memory stalls $= 2 + 3.44 = 5.44$
  - Perfect cache/Ideal CPU is $^{5.44}/_{2} = 2.72$ times faster
  - If base CPI was 1, a perfect system cache would be $^{1+3.44}/_{1} = 4.44$ times faster

```
CPI(including stalls) =  CPI(without stalls) +
(No.of Stalled cyles / Instructions)

=> CPI(including stalls) = 2 + (3.44I/I)
=>CPI(including stalls) = 5.44

As the CPI of the CPU increases, the miss penalty
remains constant. Since the clock speed of the CPU
is greater, the number of stalls the CPU has to
undergo is greater.
```

Average Access Time

- Hit time is also an important important factor in performance measurement
- Designers sometimes use Average Memory Access Time (AMAT)

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Example

Find the AMAT for a processor with a 1 ns clock cycle time, a miss penalty of 20 clock cycles, a miss rate of 5%, and a cache access time (including hit detection) of 1 clock cycle. Assume that the read and write miss penalties are the same and ignore other write stalls.

Average Access Time

- Hit time is also an important important factor in performance measurement
- Designers sometimes use Average Memory Access Time (AMAT)

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

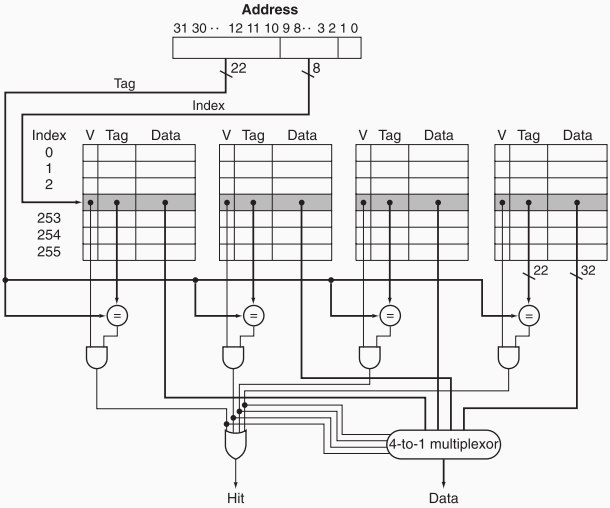
Example

Find the AMAT for a processor with a 1 ns clock cycle time, a miss penalty of 20 clock cycles, a miss rate of 5%, and a cache access time (including hit detection) of 1 clock cycle. Assume that the read and write miss penalties are the same and ignore other write stalls.

Solution:

$$AMAT = 1 + 0.05 \times 20 = 2ns\ (2 \text{ clock cycles})$$

Set Associative Cache Organization



Multilevel Cache Example I

- Given,
  - CPU base CPI $= 1$, clock-rate $= 4GHz$
  - Miss rate/instruction $= 2\%$
  - Main memory access time $= 100ns$
- Considering just the primary cache,
  - Miss penalty $= {}^{100ns}/_{0.25ns} = 400$ cycles (access time converted to cycles using clock-rate)
  - Effective CPI $= 1 + 2\% \times 400 = 9$

Multilevel Cache Example II

- Now consider an added L2 cache with,
  - Access time $= 5ns$
  - Global miss rate to main memory $= 0.5\%$
  - Main memory access time $= 100ns$
- Now, primary (L1) miss with L2 hit,
  - $\text{Penalty}_{\text{L1}\rightarrow\text{L2}} = {}^{5ns}/_{0.25ns} = 20$ cycles
- And, primary (L1) miss with L2 miss,
  - $\text{Penalty}_{\text{L2}\rightarrow\text{Mem}} = 400$ cycles (previously calculated)
- Hence, effective CPI with L2 cache

  $= 1 + \text{Primary stalls per instruction} + \text{Secondary stalls per instruction}$

  $= 1 + 2\% \times 20 + 0.5\% \times 400 = 3.4$
- Performance ratio between without/with secondary cache $= {}^{9}/_{3.4} = 2.6$

Example: MTTF vs AFR

- Assume disk MTTF is 1,000,000 hours (114 years)
- Assume warehouse-scale computer with 50,000 servers (100,000 disks)
- 1 year equals 8760 hours
- AFR is $^{8760}/_{1000000} = 0.876\%$
- Expected failures equal $100,000 \times 0.00876 = 876$ disks per year
- This averages to more than two disk failures per day

Availability and MTTR

- MTTR (Mean Time To Repair) measures service interruption
- MTBF (Mean Time Between Failures) is the sum of MTTF and MTTR
- Availability quantifies service accomplishment alternation

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

- Shrinking MTTR improves availability as much as increasing MTTF
- Availability is measured in nines (for example 5 nines is 99.999%, about 5 minutes of repair per year)